Práctica 1: Data Lake con Delta y limpieza de datos

OBJETIVOS

- Consolidar el uso de PySpark en modo local, sin desplegar ningún clúster, para ingesta, limpieza e integración de datos.
- Diseñar un data lake por capas (bronze, silver, gold) con el formato Delta Lake sobre ficheros locales.
- Integrar dos fuentes de datos relacionadas cruzándolas para enriquecer el análisis.
- Buscar y limpiar datos con defectos reales.
- Escribir y leer en Delta con particionado, entender el reparto de los datos (data skew) y equilibrarlo.
- Usar operaciones transaccionales (MERGE, DELETE) y recuperar versiones anteriores con el time travel de Delta.
- Distinguir hecho y dimensión de cara al modelo en estrella, y publicar la capa gold en MySQL por JDBC, lista para Power BI en la Práctica 2.

CONTEXTO:

A lo largo del curso vamos a construir una aplicación, que irá ganando funcionalidades prácticas a práctica. No es un conjunto de ejercicios sueltos, sino un sistema de datos completo sobre un mismo dominio (la bolsa española).

La capacidad de procesar y analizar grandes volúmenes de datos en tiempo real se ha convertido en un factor crítico para la toma de decisiones informada. En el ámbito bursátil, comprender con rapidez lo que ocurre en el mercado permite reaccionar ante cambios de volatilidad, rotaciones sectoriales y eventos corporativos.

A pesar de los esfuerzos continuos de los reguladores y del propio mercado por mejorar la transparencia, liquidez y protección del inversor, la volatilidad y los episodios de corrección siguen condicionando el comportamiento del IBEX 35, principal referencia bursátil de España. En 2022–2024, el índice ha atravesado tramos de fuertes subidas y retrocesos, poniendo de manifiesto la necesidad de comprender con mayor profundidad las tendencias y los factores que explican su evolución. Los datos muestran patrones diferenciados por sectores (finanzas, energía, utilities, telecomunicaciones, turismo, inmobiliario), lo que ha llevado a los agentes del mercado a impulsar una nueva iniciativa de información y educación financiera para el público inversor.

Con el fin de responder a este reto, se plantea una iniciativa de análisis que permita identificar tendencias, riesgos y oportunidades a distintos horizontes temporales (intradiario, semanal, mensual y anual).

En bolsa, el medio plazo generalmente abarca un horizonte temporal de uno a cinco años, buscando un equilibrio entre riesgo y rentabilidad, y se caracteriza por una mayor volatilidad aceptable en el corto plazo, permitiendo a los inversores beneficiarse de revalorizaciones moderadas, pero con mayor potencial.

Esta primera práctica construye el cimiento de datos de la aplicación y gira en torno a dos habilidades de la ingeniería de datos: limpiar e integrar. Se parte de dos fuentes distintas pero relacionadas, y el objetivo es dejarlas cruzadas y listas para analizar.

La primera fuente son los precios diarios del IBEX-35, que se descargan de Yahoo Finance con la librería yfinance. Esta información generalmente llega ya depurado. La segunda es un índice de sentimiento por sector. Para ello se facilita un CSV que se entrega con la práctica y que recoge el expertis de una persona que se dedica al análisis de la bolsa. La finalidad será averiguar si el sentimiento de un sector guarda relación con el retorno de sus empresas.

Para la entrega de la práctica es necesario entregar EL PROYECTO COMPLETO con el código ordenado, bien estructurado y funcionando. **La normativa y pautas de la primera parte son las mismas que para la practica 0.**

CLASE 1

PREPARACIÓN DEL ENTORNO

Esta práctica actualiza el stack respecto a la Práctica 0. Para trabajar con Delta Lake moderno es necesario Apache Spark 4.x y Java 17, ya que las versiones recientes de Delta han retirado el soporte de Spark 3.5. Para ello, es necesario descargar el entorno ya preparado que está en el Moodle e importarlo en Anaconda. El entorno YA ESTA CREADO con lo siguiente:

```
conda create --name aplicacion2026 python=3.11
conda activate aplicacion2026
conda install -c conda-forge openjdk=17
pip install pyspark==4.0.1 delta-spark==4.0.1
pip install yfinance
```

Adicionalmente, es necesario instalar el winutils.exe y hadoop.dll EN WINDOWS.

Winutils

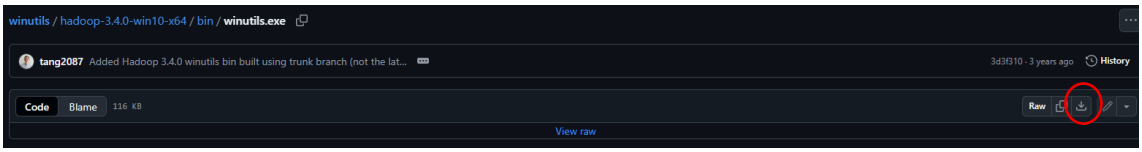
Se utiliza principalmente como un componente necesario cuando se ejecutan ciertas aplicaciones de Hadoop o frameworks que dependen de Hadoop, como Apache Spark, en sistemas operativos Windows. Instalación:

- Paso 1: **(Si quieres saltar este paso, descarga la carpeta "winutils" del Moodle y copiar los archivos)**. Para instalar winutils, hay que descargar el archivo winutils.exe correspondiente a la versión de Hadoop desde un repositorio de GitHub.

Repositorio: https://github.com/kontext-tech/winutils

Versión: Para nuestra instalación: hadoop-3.4.0-win10-x64/bin

Descargar: winutils.exe y hadoop.dll



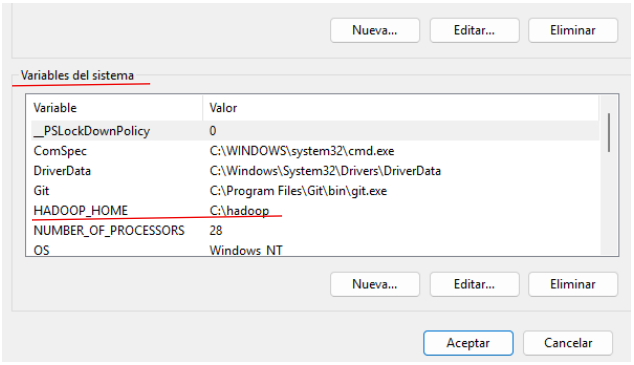
– Paso 2: En **WINDOWS** crear una nueva carpeta llamada hadoop en C.

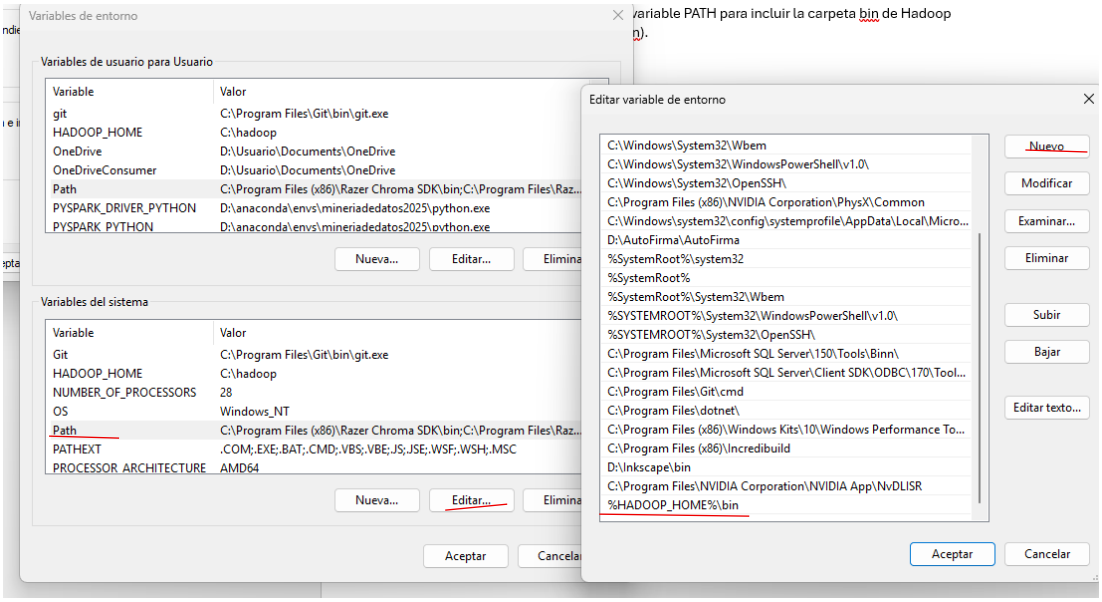
C:\hadoop

- Paso 3: Dentro de hadoop, crear una nueva carpeta llamada bin y pegar los archivos dentro:

C:\hadoop\bin

– Paso 4: Buscar en el buscador de Windows “variables de entorno”. Crear una variable de entorno llamada HADOOP_HOME apuntando al directorio de Hadoop (C:\hadoop) y EDITAR la variable PATH para incluir la carpeta bin de Hadoop (%HADOOP_HOME%\bin). Ver imágenes.





- Paso 5: Reiniciar.

SOLO EN CASO DE QUE EXISTA UN ERROR O NO LO ENCUENTRE:

ABRIR ANACONDA Y EL ENTORNO CORRESPONDIENTE. Ejecutar en consola:

```
setx HADOOP_HOME "C:\hadoop"
setx PATH "%PATH%;C:\hadoop\bin"
```

PRÁCTICA:

Yahoo finance es una librería Python que consulta datos públicos para descargar históricos dividendos, splits, etc., para acciones, índices, ETFs y divisas. La libreria **yfinance** permite descargar el histórico de una acción. Para un único valor se usa Ticker(...).history(...), que devuelve un DataFrame de **pandas** con una fila por sesión:

```
import yfinance as yf
pdf = yf.Ticker("BBVA.MC").history(start="2021-10-01", end="2026-10-01",
                    interval="1d", auto_adjust=False)
```

El código anterior, permite descargar datos con yfinance para el ticker indicado, entre start y end, con la granularidad *interva*l (por ejemplo: "1d", "1h", "15m"...).

Devuelve un DataFrame de pandas donde el índice suele ser la fecha/hora (Date/DatetimeIndex) y las columnas son Open, High, Low, Close, Volume.

En pandas, cada DataFrame tiene un índice: una etiqueta para cada fila, como si fuera su "nombre". Normalmente es un número (0, 1, 2...), pero puede ser cualquier cosa. yfinance usa la fecha como índice: cada fila está etiquetada por su día, en lugar de tener una columna Date normal.

Esto es un problema al pasar los datos a Spark, porque Spark no tiene el concepto de índice: solo entiende de columnas. Si le pasaras el DataFrame tal cual, las fechas (que están en el índice) se perderían.

La solución es reset_index(), que convierte el índice en una columna (llamada Date) y coloca un índice numérico nuevo. Así la fecha pasa a ser una columna más, que Spark sí conserva.

```
df.reset_index(inplace=True)  # Convertir el índice 'Date' a columna
```

Y posteriormente se puede crear un dataframe de spark con todos los datos. Los datos recibidos son:

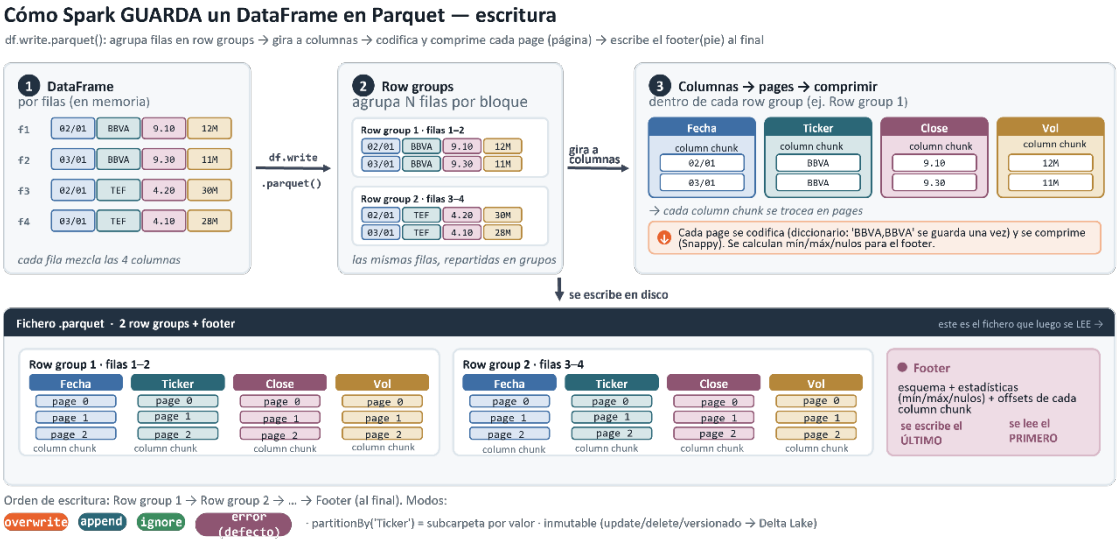
- Fecha — date, no nulo. Día de la sesión (formato YYYY-MM-DD).
- Open — double. Precio de apertura.
- High — double. Máximo del día.
- Low — double. Mínimo del día.
- Close — double. Precio de cierre.
- Volume — int, nulo permitido. Acciones negociadas en el día.
- Ticker — string, no nulo. Símbolo único que representa un activo financiero.
- Dividends — double. Dividendo pagado ese día (0.0 si no hay).

- Stock Splits — double. Ratio de desdoblamiento ese día (0.0 si no hay)

Posteriormente Convierte el DataFrame de pandas (pdf) en un DataFrame de Spark (df). Spark infiere los tipos a partir de los de pandas (los float pasan a double, los int a long, las fechas a date, etc.).

```
df = spark.createDataFrame(pdf)
```

Parquet es un formato de almacenamiento **orientado a columnas** (columnar) diseñado para análisis de datos a gran escala. A diferencia de los formatos por filas (como CSV), Parquet guarda **juntos** los valores de cada columna, lo que permite **reducir** drásticamente el tamaño en disco y el I/O. Además, soporta anidamiento (arrays, structs, mapas) y tipos complejos, lo que lo hace robusto para datos semiestructurados.



En la imagen se observan las piezas que emplea Parquet para guardar los datos por dentro. Van anidadas,

La estructura de un fichero Parquet:

- Row group (grupo de filas: Es un **corte horizontal** del fichero: un bloque con un subconjunto de las filas (por defecto ~128 MB de datos). Dentro de un row group están todas las columnas, pero solo para esas filas. Es la unidad que permite hacer predicate pushdown, es decir, como el footer guarda el mín/máx de cada columna en cada row group, Spark puede saltarse row groups enteros que no cumplen un filtro sin abrirlos (filtras Close > 100 y un row group con máx = 80 ni se lee).

-Column chunk (fragmento de columna): Dentro de cada row group, los datos se parten por columna: cada *column chunk* guarda, de forma contigua, todos los valores de UNA columna para las filas de ese row group. Esto es lo que hace posible el column pruning, es decir, si se pide Ticker y Close, Spark lee únicamente esos column chunks y se salta los de Open, High, Volume... Cada column chunk tiene además sus propias estadísticas (mín, máx, nº de nulos) en el footer.

-Snappy (compresión): Es el algoritmo de compresión que Parquet aplica a cada trozo de datos, y es el que usa Spark por defecto. Su característica es que prioriza la velocidad de comprimir/descomprimir sobre el máximo ahorro de espacio

Juntando los tres: el row group decide **qué filas** se pueden salra, el column chunk decide **qué columnas** se leen, y Snappy hace que cada column chunk **ocupe menos** en disco. Las tres cosas reducen el I/O, que es lo que da la velocidad de Parquet.

PySpark prefiere un formato con muchas filas y pocas columnas. En el mundo de Big Data, este diseño se conoce como formato "alto" o "largo" (Tall/Long format).

- PySpark divide el trabajo entre muchos nodos. Si tienes millones de filas, Spark puede partirlas en "paquetes" (particiones) y enviarlas a diferentes computadoras para que trabajen al mismo tiempo.
- Sin límite real: Puedes procesar miles de millones de filas sin que el sistema se sature, siempre que tengas suficientes nodos.
- Sin embargo, cada columna requiere que Spark guarde metadatos (el nombre de la columna, el tipo de dato, etc.). Si hay miles de columnas, la gestión de estos metadatos consume mucha memoria RAM del nodo principal (Driver).
- Además, parquet guarda los datos por columnas. Si la tabla es muy ancha y las operaciones involucran a muchas columnas, la lectura en disco se vuelve lenta y pesada.

Si tienes los datos por su naturaleza son muy "anchos" (por ejemplo, datos de sensores donde cada minuto es una columna), la mejor práctica en PySpark es aplicar una operación unpivot (se da en teoría) para convertir esas columnas en filas antes de hacer procesos pesados.

Union

Esta función apila dos DataFrames, es decir, pone las filas del segundo debajo de las del primero. Los dos deben tener las mismas columnas. union combina por posición (las columnas deben estar en el mismo orden).

```
df_total = df1.union(df2)
```

Para **almacenar** un DataFrame de PySpark en Parquet:

```
# Almacenar (sobrescribe si ya existe)
df.write.mode("overwrite").parquet("ruta/a/destino")

# Append (añadir) en vez de sobrescribir
df.write.mode("append").parquet("ruta/a/destino")

# Particionar por una columna: crea una subcarpeta por valor
df.write.mode("overwrite").partitionBy("Ticker").parquet("ruta/a/destino")
```

Los modos de escritura disponibles son overwrite (sobrescribir), append (añadir), ignore (no hacer nada si ya existe) y error (fallar si ya existe, es el de por defecto).

Para leer datos, spark.read devuelve un DataFrame:

```
# Archivo o carpeta Parquet
df = spark.read.parquet("ruta/a/datos")

# Solo algunas columnas (column pruning)
df = spark.read.parquet("ruta").select("col1", "col2")

# Con filtro: Spark se salta los row groups que no cumplen (predicate pushdown)
df = spark.read.parquet("ruta").filter("Close > 100")
```

Un fichero Parquet es inmutable ya que no se edita en el sitio, por eso append no modifica nada, solo escribe ficheros nuevos en la carpeta. Por ese motivo Parquet, por sí solo, no permite operaciones de actualización, borrado ni control de versiones. **Delta Lake resuelve esto.**

Errores

Atención: Si se obtiene un error de este estilo:

Python worker exited unexpectedly (crashed), Connection reset y el WinError 10038 … no es un socket.

Solución:

```
import sys, os
os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable
```

EJERCICIOS

A continuación, realiza las siguientes funcionalidades. Indica claramente a que ejercicio pertenece cada sección del código tal y como se especificó en la práctica 0:

- Ej1a: Crea una función que permita descargar el histórico de una acción concreta, facilitando una fecha de inicio y una de fin, y lo almacene en un dataframe de pyspark. Define para cada columna si permite nulos y por qué.

- Ej1b: Usando la función anterior, descarga los últimos 2 años (último día 1 de octubre de 2026) de todas las empresas del IBEX-35 y guárdalo **crudo** en la capa bronce (lo veremos más adelante que significa), en formato Parquet:

```
df.write.mode("overwrite").parquet("data/lake/bronze/ibex")
```

- Ej1c (opcional): Lee de vuelta el Parquet que acabas de escribir y muestra el esquema y las 5 primeras filas. A continuación, lee **solo las columnas** Ticker, Date y Close (column pruning) y muéstralas. Comenta qué ventaja tiene leer solo esas columnas frente a leerlas todas.

- Ej2a: Guarda el fichero sentimiento_ibex.csv en un DataFrame de PySpark. No abrir el csv. Observa y comenta que todas las columnas llegan como texto (un CSV no guarda los tipos).

- Ej2b: Guarda el sentimiento crudo en la capa bronze en formato Parquet, en su propia carpeta (data/lake/bronze/sentimiento). Muestra cuántas filas se han escrito.

- Ej2c (opcional): Vuelve a ejecutar la carga del Ej2b pero esta vez en modo append. Cuenta las filas de la carpeta antes y después y comprueba que pasa. Comenta que ocurre.

CLASE 2

Delta Lake: es una evolución del almacenamiento en ficheros para tablas de datos, que fusiona técnicas de los *data lakes* y de los *data warehouse* (se dan en teoría). Toma la tecnología Parquet actual y le añade un **registro de transacciones** (es decir, *transaction log*) que crea transacciones **ACID**, como las de una base de datos. Es decir, los datos siguen siendo ficheros Parquet pero ahora hay un log encima que coordina todo lo que se escribe. Por ejemplo, si se está escribiendo 10 ficheros Parquet y a mitad se cae la luz no se sabe si los datos están completos o corruptos. Y si dos procesos escriben a la vez, pueden pisarse. Las cuatro garantías ACID son:

- Atomicidad: Las operaciones de una transacción ocurren como un bloque único; o se ejecutan todas con éxito, o no se realiza ninguna.
- Consistencia: Toda transacción lleva a la base de datos de un estado válido a otro, cumpliendo siempre con las reglas y restricciones definidas.
- Aislamiento: Las transacciones que se ejecutan de forma simultánea no interfieren entre sí, evitando que datos en procesos intermedios sean visibles.
- Durabilidad: Una vez que una transacción finaliza correctamente, los cambios se guardan de forma permanente, incluso si ocurre un fallo en el sistema.

Crear la sesión de Spark con Delta

Para trabajar con Spark hay que crear una SparkSession. Para a usar Delta Lake, la sesión necesita una configuración extra que lo active.

```
from pyspark.sql import SparkSession
from delta import configure_spark_with_delta_pip

builder = (SparkSession.builder
        .appName("IBEX35")
        .master("local[*]")
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog",
            "org.apache.spark.sql.delta.catalog.DeltaCatalog"))

spark = configure_spark_with_delta_pip(builder).getOrCreate()
```

Cada una de las partes

- SparkSession.builder. El "constructor" de la sesión. Sobre él se van encadenando las opciones de configuración.
- .config("spark.sql.extensions", "...DeltaSparkSessionExtension"). Activa las extensiones de Delta para que Spark entienda sus operaciones (MERGE, time travel, etc.). Obligatorio para usar Delta.
- .config("spark.sql.catalog.spark_catalog", "...DeltaCatalog"). Hace que el catálogo de tablas de Spark use Delta. Obligatorio para usar Delta.
- configure_spark_with_delta_pip(builder). Añade automáticamente los .jar de Delta que Spark necesita (al haber instalado delta-spark con pip). Obligatorio.
- .getOrCreate() — crea la sesión (o reutiliza una ya existente) y devuelve el objeto spark, que es con el que se trabaja en toda la práctica.

Transaction log (registro de transacciones)

Este registro es lo que hace que escribir en una tabla sea una operación atómica, es decir, o se escribe entera o no se escribe nada y que dos procesos puedan trabajar sobre la misma tabla sin corromperla. El componente central es el registro de transacciones, un log ordenado de cada transacción hecha sobre la tabla. Este componente habilita **seis acciones**: añadir fichero, eliminar fichero, actualizar metadatos, fijar una transacción, cambiar el protocolo y confirmar información. (Añadir un fichero nuevo equivale a hacer *append*.) Se guarda en la carpeta _delta_log, con ficheros JSON de *commit*; el primero es 000000.json. Recorrer ese log es, precisamente, lo que permite todo lo que viene a continuación: deshacer, actualizar y volver a versiones anteriores de la base de datos. Por tanto, no guarda una copia entera cada vez que se cambia algo, se almacena qué cambió respecto al estado anterior. Para saber cómo está la tabla "ahora", Spark lee el log de principio a fin y reconstruye el estado.

Operaciones que habilita Delta

- **Convertir Parquet a Delta.** Como una tabla Delta *son* ficheros Parquet más el log, una carpeta Parquet que ya exista se puede convertir en tabla Delta sin reescribir los datos: solo se le añade el _delta_log. Es una operación de metadatos, muy rápida. CUIDADO CON LAS COMILLAS PARA LA RUTA (backticks).

```
from delta.tables import DeltaTable
DeltaTable.convertToDelta(spark, "parquet.Restore` data/lake/bronze/ibex ` ")
```

- **Escribir y leer en Delta.**

```
df.write.format("delta").mode("overwrite").save("data/lake/bronze/ibex")
df = spark.read.format("delta").load("data/lake/bronze/ibex")
```

- **Validación y evolución de esquema**. Por defecto Delta aplica schema enforcement (validación de esquema en la escritura), es decir, si intentas añadir datos cuyas columnas no coinciden exactamente con las de la tabla (por ejemplo, una columna de más), la operación se rechaza con un error de esquema. Esto

protege la tabla de escrituras accidentales que la dejarían inconsistente. Cuando el cambio sí es intencionado, se activa la evolución de esquema con la opción mergeSchema, que permite que la nueva columna se incorpore a la tabla y las filas antiguas quedan con valor nulo en esa columna.

```
# Falla: la tabla no tiene la columna nueva (schema enforcement)
df_nuevo.write.format("delta").mode("append").save("data/lake/bronze/ibex")

# Funciona: Delta añade la columna al esquema (schema evolution)
(df_nuevo.write.format("delta").mode("append")
      .option("mergeSchema", "true")
      .save("data/lake/bronze/ibex"))
```

- **Merge.** Combina datos nuevos con una tabla existente en una sola operación: actualiza las filas que coinciden por una clave (es decir, ***whenMatchedUpdate*** permite modificar cuando esta fila ya existe en destino y se quiere actualízar con los datos nuevos.) o insertar las que no existen (es decir, ***whenNotMatchedInsert*** permite insertar filas cuando no existe en destino). Es la forma de conseguir una carga **idempotente** (es decir, ejecutarla dos veces no duplica datos, pero se pueden añadir nuevos). Cruza **DOS tablas** (destino y una **tabla nueva de fuera)**. Compara fila a fila por una clave y decide qué hacer con cada una (insertar, actualizar, o ignorar). Aquí es donde Delta resuelve el problema de Parquet.

```
destino = DeltaTable.forPath(spark, "data/lake/bronze/ibex")

(destino.alias("d")

  .merge(nuevos.alias("n"), "d.Ticker = n.Ticker AND d.Fecha = n.Fecha")

  .whenNotMatchedInsertAll()

  .execute())
```

El matiz de Insert vs InsertAll

- whenNotMatchedInsertAll() → inserta la fila entera tal cual viene (todas las columnas). Es el atajo cómodo cuando la estructura coincide.
- whenNotMatchedInsert({...}) → le dices columna por columna qué poner. Lo usas si quieres mapear nombres distintos o rellenar solo algunas.

- **Delete.** Borra filas según una condición, sin reescribir el resto de la tabla:

```
destino.delete("Ticker = 'UNI'")
```

- **Update.** Actualiza filas por condición, como el DELETE pero cambiando valores (destino.update(condition=..., set={...})). **Solo trabaja con la tabla que ya esta (UNA TABLA)**.

```
destino.update(condition="Ticker = 'UNI'", set={"Close": "0"})
```

- **Time travel.** Como el log guarda el historial de versiones, se puede consultar una versión anterior de la tabla, útil para recuperar datos tras un borrado o una carga errónea. El método history() devuelve un DataFrame con un registro por cada versión de la tabla, ordenado de la más reciente a la más antigua. Pasándole un número, history(1) devuelve solo el último commit, es decir, la versión actual.

```
    DeltaTable.forPath(spark, "data/lake/bronze/ibex").history().show()
df_v0=spark.read.format("delta").option("versionAsOf",0).load("data/lake/bronze/ibex")
```

A veces no basta con mostrarlo por pantalla, **sino que se necesita el número de versión dentro del propio programa** (por ejemplo, para guardarlo y volver a él más tarde). NO CAMBIA NADA. Para extraerlo se selecciona la columna version y se trae ese único valor al driver con collect():

```
    # Número de la versión actual (un entero)
    v = DeltaTable.forPath(spark, "data/lake/bronze/ibex") \
        .history(1).select("version").collect()[0][0]
```

- **Restore.** Hay que distinguir dos cosas. *versionAsOf* solo lee una versión antigua. Esta función demuestra que el dato sigue siendo recuperable, pero no toca la tabla, que sigue en su estado actual. Si lo que se quiere es dejar la tabla realmente como estaba en una versión anterior (por ejemplo, para deshacer una carga errónea), se usa restoreToVersion(). RESTORE no borra el historial si no que crea un commit nuevo cuyo contenido es idéntico al de la versión indicada, de modo que la tabla vuelve a ese estado pero el log conserva todo lo ocurrido (incluido el propio RESTORE).

```
    tabla = DeltaTable.forPath(spark, "data/lake/bronze/ibex")

    # Solo LEE la versión 3 (la tabla actual no cambia)
    df_v3 = spark.read.format("delta").option("versionAsOf",
    3).load("data/lake/bronze/ibex")

    # DEJA la tabla como estaba en la versión 3 (crea una versión nueva)
    tabla.restoreToVersion(3)
```

- **Mantenimiento.** Con el tiempo conviene compactar y limpiar la tabla: OPTIMIZE junta los ficheros pequeños para leer más rápido, y VACUUM borra los ficheros antiguos que ya no forman parte de ninguna versión reciente (por defecto conserva los de los últimos 7 días). Conviene tener claro que el time travel **no** es una copia

de seguridad: sirve para revisar cambios recientes, no para archivar datos a largo plazo.

Otras capacidades

Delta ofrece más funciones que no se usan en esta práctica, pero conviene conocer:

- **Clones** (superficiales o profundos): copias de una tabla para pruebas o respaldo.

- **Change data feed (CDC)**: un registro aparte de los cambios hechos sobre una tabla.

- **Z-ordering**: reordena físicamente los datos de una columna muy filtrada para saltarse ficheros enteros al leer.

- **Bloom filters**: un índice probabilístico que descarta rápido los ficheros que no contienen un valor buscado.

Sobre todas las funcionalidades de delta, realiza los siguientes ejercicios.

- Ej3: Convierte las carpetas de bronze (ibex y sentimiento), que ahora son Parquet, en tablas Delta, sin reescribir los datos. Muestra el historial y comenta qué ha aparecido en la carpeta (_delta_log).

- Ej4: Visualiza la versión actual de la base de datos. Ahora vamos a incluir datos nuevos. Vuelve a descargar los precios del IBEX-35 pero hasta hoy. Haz MERGE sobre la clave (Ticker, Fecha):
    - En la primera ejecución, comprueba que se insertan los días nuevos (el número de filas sube).
    - Ejecútalo una segunda vez y comprueba que pasa.

- Ej5a: Crea una columna nueva, con los datos de la columna close y sumale 5 (nombre de la columna Close_mas5). Intenta escribir la tabla con esa columna de más en modo append y comprueba que Delta lo rechaza (error de esquema).

- Ej5b: Repite la escritura del ejercicio anterior con option("mergeSchema", "true") y comprueba si sigue rechazándolo.

- Ej5c: Elimina con DELETE las filas del ticker BBVA. Muestra las filas antes y después.

- Ej5d: Muestra el history() (verás las versiones de los Ej5b y Ej5c) y lee con versionAsOf a la versión de antes del Ej5a. Comprueba si todo esta igual.

- Ej6: VersionAsOf lee la versión antigua (demuestra que el dato sigue recuperable), pero no cambia la tabla actual. Deja la tabla realmente como estaba en el EJ4, usando RESTORE.

CLASE 3

En esta clase se construyen las dos capas que faltan del medallón: silver (datos limpios y tipados) y gold (las dos fuentes integradas), y se preparan los datos para que Power BI los use en la Práctica 2.

La arquitectura medallion (bronze, silver, gold): Un data lake no es un montón de ficheros sueltos si no que se organiza en **capas de calidad creciente**. El patrón más habitual en ingeniería de datos es la *arquitectura medallion*, que ordena los datos en tres niveles: **bronze, silver y gold**. La idea de fondo es que cada capa tiene una responsabilidad clara y que nunca se mezcla el dato crudo con el ya procesado: así, si algo sale mal, siempre puedes reprocesar desde la capa anterior sin volver a la fuente.

- Bronze (crudo**).** Es una copia fiel de cada fuente, tal y como llega, sin transformar. Su valor es ser el registro original: permite reprocesar sin tener que volver a descargar de Yahoo ni a leer otra vez el CSV. En esta práctica, cada fuente va en **su propia carpeta** dentro de bronze (los precios por un lado, el sentimiento por otro), porque tienen esquemas distintos y todavía no se cruzan.

- Silver (limpio). Es el dato ya limpio, tipado y validado. Aquí se corrigen errores, se ajustan los tipos (fechas, números) y se eliminan duplicados.

- Gold (listo para consumir). Es el dato enriquecido y agregado, preparado para el análisis y para las herramientas de negocio.

Bronze, silver y gold son solo una organización lógica (el medallón) por nivel de calidad. Técnicamente, las tres son tablas Delta normales, así que cualquiera de ellas admite MERGE, DELETE, UPDATE, time travel, mergeSchema, OPTIMIZE, etc.

La capa Bronze, corresponde con los ejercicios realizados en la Clase 1 y la Clase 2. Como se ha visto, los precios de Yahoo vienen bastante limpios, pero el CSV de sentimiento **llega sucio y en formato ancho (si no sabes lo que es, revisa el concepto)** y con defectos reales: decimales con coma mezclados con punto, celdas vacías y nombres de sector en español. La capa silver deja cada fuente tipada, normalizada y sin duplicados. La mayoría de las herramientas para limpiar estos datos se vieron en la Práctica 0.

Silver: limpiar, tipar y validar

El sector de cada empresa (desde Yahoo): Para cruzar los precios con el sentimiento hace falta saber a qué sector pertenece cada empresa. Ese dato no viene en el histórico de precios, pero Yahoo lo expone en el objeto info de cada ticker:

Estas son las funciones más útiles para limpiar los datos.

- Leer el CSV usando el separador ; y cabecera.
  spark.read.option("header", True).option("sep", ";")

- Unificar , y . para los valores decimales.
  regexp_replace

- Cambiar de texto a date
  F.to_date("fecha")
- Tratar los nulos
  dropna()

- Quitar duplicados
  dropDuplicates()

- **Unpivot**: La operación (también conocida como melt o "despivotar") es un proceso de transformación de datos que cambia la estructura de una tabla, pasándola de un formato "ancho" (wide) a un formato "largo" (long). Su objetivo principal es **reunir varias columnas similares y convertirlas en filas**, haciendo que los datos sean más limpios, estandarizados y fáciles de analizar.

```
df_largo = df_ancho.unpivot(ids, values, variableColumnName, valueColumnName)
```

1. **ds (Columnas Identificadoras):**
   - Qué recibe: Una columna, una lista de nombres de columnas (["col1", "col2"]) o expresiones de Spark.
   - Qué hace: Define qué columnas permanecerán fijas e intactas en la tabla. Sus valores no se mueven, simplemente se repiten hacia abajo para acompañar a las nuevas filas generadas.

2. **values (Columnas a Despivotar):**
   - Qué recibe: Una lista de las columnas que quieres transformar/colapsar, o None (si pasas None, Spark tomará automáticamente todas las columnas que no hayas puesto en el parámetro ids).
   - Qué hace: Son las columnas que van a "desaparecer" en el sentido horizontal y se van a convertir en filas. *Nota: Todas las columnas en esta lista deben tener tipos de datos compatibles.*

3. **variableColumnName (Nombre de la columna de nombres):**
   - Qué recibe: Un string (cadena de texto).
   - Qué hace: Define el nombre de la nueva columna que se creará para almacenar los *nombres antiguos* de las columnas que despivotaste (los encabezados).

4. **valueColumnName (Nombre de la columna de valores):**
   - Qué recibe: Un string (cadena de texto).
   - Qué hace: Define el nombre de la nueva columna que se creará para almacenar los *datos o métricas* que estaban dentro de las celdas de la tabla original.

- **Anti-join (left_anti)** el join normal (inner/left/right/full) ya se vio en la P0, el anti-join devuelve las filas de la izquierda que no tienen pareja en la derecha, y sirve para ver qué no cruza. Imagina que tienes dos listas, **(A):** Todos tus clientes registrados en la plataforma. **(B):** Clientes que realizaron una compra este mes. Si aplicas un **left_anti** uniendo ambas tablas por el ID_Cliente, el resultado será los Clientes que NO han comprado nada este mes. (Es decir, los elementos de A que no tienen ninguna pareja ni correspondencia en B).

```
df_resultado = df_izquierda.join(df_derecha, on=columnas_clave, how="left_anti")
```

1. **other (Primer parámetro / DataFrame Derecho):**
    - Qué recibe: El DataFrame de la derecha (df_derecha).
    - Qué hace: Es la tabla que se usará como filtro de comparación. Spark buscará sus claves para saber cuáles descartar de la tabla izquierda.

2. **on (Columnas Clave):**
    - Qué recibe: Un string ("id"), una lista de strings (["Sector", "fecha"]), o una condición lógica armada con expresiones (df_izq.id == df_der.id).
    - Qué hace: Define la condición o las columnas que deben coincidir en ambas tablas para que una fila sea considerada "emparejada" (y por lo tanto, eliminada del resultado).

3. **how (Tipo de Join):**
    - Qué recibe: Un string. Para este caso exacto debe ser "left_anti" (también acepta "anti").
    - Qué hace: Le indica al motor de Spark que ejecute la lógica de exclusión: conservar solo lo que está a la izquierda y que NO tiene coincidencia en la derecha.

Gold: limpiar, tipar y validar

La capa gold es la capa de consumo donde los datos dejan de estar organizados por su origen y pasan a organizarse por las preguntas que se les van a hacer. Esa es la diferencia esencial con silver. En bronze y en silver los datos conservan la forma de cada fuente (los precios por un lado, el sentimiento por otro, cada uno limpio, pero independiente). En gold se cruzan y se modelan pensando en el análisis y en las herramientas de negocio que los van a leer. Dicho de otro modo, silver está orientado a ser fiel al dato de origen, y gold está orientado a responder preguntas rápido.

Para ese fin, el modelado analítico separa la información en dos tipos de tabla: hechos y dimensiones.

- El hecho (fact). Una tabla de hechos registra eventos medibles que han ocurrido: cada fila es un evento. En esta práctica, el evento es “la cotización de una empresa en un día”, enriquecida con el sentimiento de su sector.

- Las dimensiones (dimension). Una dimensión aporta el contexto descriptivo que da sentido a las medidas: el “quién, qué y cuándo”. Son, sobre todo, atributos de

texto que cambian poco o nada con el tiempo y que sirven para filtrar, agrupar y etiquetar los hechos, se usan como criterios. En esta práctica hay dos dimensiones naturales. La empresa (Tiker) y la Fecha.

La prueba para distinguirlas es sencilla: si un dato lo quieres sumar o promediar, es una medida (hecho); si lo quieres usar para filtrar o agrupar, es un atributo de dimensión.

El modelo en estrella. Cuando se coloca la tabla de hechos en el centro y se la rodea de sus dimensiones, unidas por sus claves (Ticker, Sector, Fecha), el diagrama recuerda a una estrella.

Por eso el modelo en estrella es el que las herramientas de inteligencia de negocio, como Power BI, entienden de forma nativa: cada dimensión se convierte en filtros y ejes, y las medidas del hecho se agregan al vuelo según esos filtros. Por tanto, la dimension gold no se almacena como una única tabla, sino como un conjunto de tablas relacionadas que forman el modelo en estrella: una tabla de hechos y una tabla por cada dimensión. Cada una es una tabla Delta independiente, en su propia carpeta dentro de data/lake/gold.

Sobre todas las funcionalidades de Delta, realiza los siguientes ejercicios. Junto con el código, es necesario entregar un PDF breve que recoja las decisiones tomadas en cada ejercicio: qué función se ha usado, por qué, y cómo se interpreta el resultado.

Cómo estructurar el código: Los ejercicios no se resuelven metiendo todo el código dentro de un único método (Ej1, Ej2...). Hay que seguir el patrón MVC que usamos en la aplicación: la lógica se escribe como funciones reutilizables en el modelo —cada una resuelve un problema concreto (descargar, limpiar, convertir a Delta, hacer MERGE, cruzar, exportar...)— y cada ejercicio se limita a llamar a esas funciones y mostrar la comprobación que se pida. Así, una misma función (por ejemplo, la que escribe en Delta o la que hace el anti-join) se escribe una sola vez y se reutiliza en varios ejercicios, en lugar de repetir código.

- Ej7: Analiza los datos e incluye una reflexión de como podrías tener en cuenta la información del experto en los datos que puedes obtener de yahoo finance. (Investiga que datos tiene la API de yahoo u otras interesantes). Justifícalo en el pdf
- Ej8: Construye la capa silver. Limpia la base de datos sentimiento y guárdalo en data/lake/silver/sentimiento**. Los sectores del CSV están en español**. Justifica la construcción/modificación de los datos en el pdf. Muestra cuántas filas tiene cada tabla.
- Ej9: Deja los datos de la api de yahoo finance limpos (si es necesario) y guárdalos en data/lake/silver/ibex. Si necesitas más datos (Ej7) guarda en una dimensión (o varias) aparte tanto en bronce (datos en crudo) como en silver (datos corregidos) la información. Muestra la/s fila/s 5.
- Ej10: Integra todas las fuentes en la capa y guárdala en data/lake/gold.

- Ej11 (opcional): Con un anti-join, muestra (a) los precios que se quedan sin sentimiento y (b) el sentimiento sin precios. Comenta que sucede y como lo solucionas.
- Ej12 (opcional): Vuelve al estudio realizado en el Ej7 y piensa que datos te gustaría ver en el dashboard (adicionales a los datos tal cual) que se hará en power BI (por ejemplo, la variación de precio respecto al día anterior). Realiza al menos dos operaciones/cálculos y guárdalos en una nueva columna.
- Ej13: Exporta la gold a Parquet en data/export/gold, lista para Power BI. (Tienes que elegir como lo vas a guardar). Investiga más sobre la forma de estrella para guardar los datos (Cuanto más eficiente lo hagas, mejor funcionará la siguiente práctica. Pregunta a tu profesor si tienes dudas una vez tengas el modelo). Justifícalo en UN párrafo en el pdf.